

eye

Identity matrix

Syntax

```
I = eye
I = eye(n)
I = eye(n,m)
I = eye(sz)

I = eye( __ ,typename)
I = eye( __ ,like=p)
```

Description

I = **eye** returns the scalar 1.

I = **eye**(**n**) returns an n-by-n identity matrix with ones on the main diagonal and zeros elsewhere. [example](#)

I = **eye**(**n**,**m**) returns an n-by-m matrix with ones on the main diagonal and zeros elsewhere. [example](#)

I = **eye**(**sz**) returns an array with ones on the main diagonal and zeros elsewhere. The size vector, **sz**, defines **size(I)**. For example, **eye**([2,3]) returns a 2-by-3 array with ones on the main diagonal and zeros elsewhere. [example](#)

I = **eye**(__ ,**typename**) also specifies the data type (class) of **I** for any of the previous syntaxes. For example, **eye**(5,"int8") returns a 5-by-5 identity matrix consisting of 8-bit integers. [example](#)

I = **eye**(__ ,**like**=**p**) specifies that **I** has the same data type, sparsity, and complexity (real or complex) as the numeric variable **p**. [example](#)

Examples

[collapse all](#)

Square Identity Matrix

Create a 4-by-4 identity matrix.

[Open in MATLAB Online](#)[Copy Command](#)

```
I = eye(4)
```

[Get](#) ▼

I = 4×4

1	0	0	0
0	1	0	0
0	0	1	0

$$\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

Rectangular Matrix

Create a 2-by-3 identity matrix.

[Open in MATLAB Online](#)[Copy Command](#)

```
I = eye(2,3)
```

[Get](#) ▼

```
I = 2×3
```

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Identity Vector

Create a 3-by-1 identity vector.

[Open in MATLAB Online](#)[Copy Command](#)

```
sz = [3,1];  
I = eye(sz)
```

[Get](#) ▼

```
I = 3×1
```

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Nondefault Numeric Data Type

Create a 3-by-3 identity matrix whose elements are 32-bit unsigned integers.

[Open in MATLAB Online](#)[Copy Command](#)

```
I = eye(3, "uint32")
```

[Get](#) ▼

```
I = 3x3 uint32 matrix
```

```
1  0  0
0  1  0
0  0  1
```

```
class(I)
```

[Get](#)

```
ans =
'uint32'
```

Complex Identity Matrix

Create a 2-by-2 identity matrix that is not real valued, but instead is complex like an existing array.

Define a complex vector.

[Open in MATLAB Online](#)
[Copy Command](#)

```
p = [1+2i 3i];
```

[Get](#)

Create an identity matrix that is complex like p.

```
I = eye(2,like=p)
```

[Get](#)

```
I = 2x2 complex
```

```
1.0000 + 0.0000i    0.0000 + 0.0000i
0.0000 + 0.0000i    1.0000 + 0.0000i
```

Sparse Identity Matrix

Define a 5-by-5 sparse matrix.

[Open in MATLAB Online](#)
[Copy Command](#)

```
p = sparse(5,5,pi);
```

[Get](#)

Create a 5-by-5 identity matrix that is sparse like P.

```
I = eye(5,like=p)
```

[Get](#)

```
I = 5x5 sparse double matrix (5 nonzeros)
```

```
(1,1)    1
(2,2)    1
(3,3)    1
```

```
(4,4)      1
(5,5)      1
```

Size and Numeric Data Type Defined by Existing Array

Define a 2-by-2 matrix of single precision.

[Open in MATLAB Online](#)[Copy Command](#)

```
p = single([1 3; 2 4]);
```

[Get](#) ▼

Create an identity matrix that is the same size and data type as p.

```
I = eye(size(p),like=p)
```

[Get](#) ▼

```
I = 2x2 single matrix
```

```
    1    0
    0    1
```

```
class(I)
```

[Get](#) ▼

```
ans =
'single'
```

Input Arguments

[collapse all](#)

n — Size of first dimension of I
integer value

Size of first dimension of I, specified as an integer value.

- If n is the only integer input argument, then I is a square n-by-n identity matrix.
- If n is 0, then I is an empty matrix.
- If n is negative, then it is treated as 0.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

m — Size of second dimension of I
integer value

Size of second dimension of I, specified as an integer value.

- If m is 0, then I is an empty matrix.
- If m is negative, then it is treated as 0.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

sz — Size of I

row vector of no more than two integer values

Size of I, specified as a row vector of no more than two integer values.

- If an element of sz is 0, then I is an empty matrix.
- If an element of sz is negative, then the element is treated as 0.

Example: sz = [2 3] defines I as a 2-by-3 matrix.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

typename — Output class

"double" (default) | "single" | "logical" | "int8" | "uint8" | ...

Output class, specified as "double", "single", "logical", "int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", or "uint64".

p — Prototype

numeric variable

Prototype, specified as a numeric variable.

Data Types: double | single | logical | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

Complex Number Support: Yes

Output Arguments

[collapse all](#)

I — Identity matrix

scalar | matrix

Identity matrix, returned as a scalar or matrix.

Extended Capabilities

[collapse all](#)

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

Usage notes and limitations:

- typename must be a built-in MATLAB® numeric type. Does not invoke the static eye method for other classes. For example, eye(m, n, 'myclass') does not invoke myclass.eye(m,n).
- Size arguments must have a fixed size.

GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Usage notes and limitations:

- typename must be a built-in MATLAB numeric type. Does not invoke the static eye method for other classes. For example, `eye(m, n, 'myclass')` does not invoke `myclass.eye(m,n)`.
- Size arguments must have a fixed size.

Thread-Based Environment

Run code in the background using MATLAB® `backgroundPool` or accelerate code with Parallel Computing Toolbox™ `ThreadPool`.

The `eye` function fully supports thread-based environments. For more information, see [Run MATLAB Functions in Thread-Based Environment](#).

GPU Arrays

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

The `eye` function supports GPU array input with these usage notes and limitations:

- You can specify `typename` as `'gpuArray'`. If you specify `typename` as `'gpuArray'`, the default underlying type of the array is `double`.

To create a GPU array with underlying type `datatype`, specify the underlying type as an additional argument before `typename`. For example, `I = eye(3,datatype,'gpuArray')` creates a 3-by-3 GPU identity matrix with underlying type `datatype`.

You can specify the underlying type `datatype` as one of these options:

- `'double'`
 - `'single'`
 - `'logical'`
 - `'int8'`
 - `'uint8'`
 - `'int16'`
 - `'uint16'`
 - `'int32'`
 - `'uint32'`
 - `'int64'`
 - `'uint64'`
- You can also specify the numeric variable `p` as a `gpuArray`.
If you specify `p` as a `gpuArray`, the underlying type of the returned array is the same as `p`.

For more information, see [Run MATLAB Functions on a GPU](#) (Parallel Computing Toolbox).

Distributed Arrays

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

The `eye` function supports distributed arrays with these usage notes and limitations:

- You can specify `typename` as `'codistributed'` or `'distributed'`. If you specify `typename` as `'codistributed'` or `'distributed'`, the default underlying type of the returned array is `double`.

To create a distributed or codistributed array with underlying type `datatype`, specify the underlying type as an additional argument before `typename`. For example, `I = eye(3,datatype,'distributed')` creates a 3-by-3 distributed identity matrix with underlying type `datatype`.

You can specify the underlying type `datatype` as one of these options:

- `'double'`
 - `'single'`
 - `'logical'`
 - `'int8'`
 - `'uint8'`
 - `'int16'`
 - `'uint16'`
 - `'int32'`
 - `'uint32'`
 - `'int64'`
 - `'uint64'`
- You can also specify `p` as a codistributed or distributed array.
If you specify `p` as a codistributed or distributed array, the underlying type of the returned array is the same as `p`.
 - For additional codistributed syntaxes, see [eye \(codistributed\)](#) (Parallel Computing Toolbox).

For more information, see [Run MATLAB Functions with Distributed Arrays](#) (Parallel Computing Toolbox).

Version History

Introduced before R2006a

See Also

[speye](#) | [ones](#) | [zeros](#)

Topics

[Class Support for Array-Creation Functions](#)

YOU MIGHT ALSO BE INTERESTED IN

[Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB](#)

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

[Learn more](#)